



MASTER INTELLIGENCE ARTIFICIELLE
TRAVAUX DIRIGÉS — MACHINE LEARNING

TD N° 06

SVM — Machines à Vecteurs

de Support : Marges et Noyaux

Durée : 3 heures	Niveau : Intermédiaire
Datasets : Synthétique 2D + BankChurners	Outils : sklearn, numpy, matplotlib, seaborn

Objectifs du TD

- Comprendre l'idée géométrique : **hyperplan séparateur à marge maximale**
- Visualiser les **vecteurs support** et la marge d'un SVM linéaire en 2D
- Analyser l'effet du paramètre **C** (soft margin) sur le biais/variance
- Appliquer le **kernel trick** : comprendre pourquoi les noyaux élèvent la dimension
- Comparer les noyaux **linéaire, RBF et polynomial** sur des données non-linéairement séparables
- Optimiser **C et gamma** par GridSearchCV — trouver le couple optimal
- Appliquer un SVM complet sur **BankChurners** et interpréter les résultats
- Introduire la **SVR** (régression) et la classification **multi-classe OvO/OvR**

Pr. BEN LAHMAR EL Habib · Semestre 2024–2025



Contexte et Rappels Théoriques

Le SVM (Support Vector Machine) est l'un des algorithmes les plus élégants du ML : là où d'autres modèles cherchent une frontière qui sépare les classes, le SVM cherche la frontière qui les sépare avec la **plus grande marge possible**. Ce principe de maximisation de marge lui confère une excellente généralisation.

📌 Rappel de cours — SVM Linéaire — Hyperplan à Marge Maximale

Hyperplan séparateur : $w \cdot x + b = 0$ | **Marges** : $w \cdot x + b = +1$ (classe +1) et $w \cdot x + b = -1$ (classe -1)

- **Marge** = distance entre les deux hyperplans marginaux = $2 / ||w||$
- **Maximiser la marge** $\Leftrightarrow$ minimiser $||w||^2$ sous les contraintes $y_i(w \cdot x_i + b) \geq 1$
- **Vecteurs support** : les exemples exactement sur les hyperplans marginaux ($y_i(w \cdot x_i + b) = 1$)
- **Propriété remarquable** : la décision ne dépend que des vecteurs support — le reste des données est ignoré
- **SVM hard margin** : aucun exemple dans la marge (données linéairement séparables uniquement)
- **SVM soft margin** (C) : tolère des violations de marge — compromis marge large vs erreurs de classif.

📌 Rappel de cours — Paramètre C — Compromis Biais/Variance

Valeur de C	Effet sur la marge	Comportement
C très petit ($\rightarrow 0$)	Marge très large, beaucoup de violations	Sous-apprentissage (biais élevé)
C = 1 (défaut)	Compromis équilibré	Bon point de départ
C très grand ($\rightarrow \infty$)	Marge très petite, peu de violations	Sur-apprentissage (variance élevée)

- C est l'inverse de la force de régularisation — grand C = moins de régularisation (analogue à $1/\lambda$ en RL)

📌 Rappel de cours — Kernel Trick — Séparation Non-Linéaire

- **Idée** : si les données ne sont pas linéairement séparables en dimension d, les projeter dans une dimension $d' \gg d$ où elles le sont
- **Astuce** : on n'a jamais besoin de calculer $\phi(x)$ explicitement — seulement $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$

Noyau linéaire : $K(x, y) = x^T y$ | **Noyau polynomial** : $K(x, y) = (\gamma \cdot x^T y + r)^d$

Noyau RBF (Gaussien) : $K(x, y) = \exp(-\gamma ||x - y||^2)$ $\rightarrow$ le plus populaire, dimension infinie

- **γ (gamma)** du noyau RBF : grand $\gamma \rightarrow$ chaque exemple n'influence que son voisinage proche $\rightarrow$ sur-apprentissage



- **Règle pratique** : commencer par RBF avec GridSearch sur C et γ

Chargez les bibliothèques et préparez les données :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC, SVR
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     cross_val_score, validation_curve)
from sklearn.metrics import (classification_report, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay,
                             roc_auc_score, roc_curve)
from sklearn.pipeline import Pipeline
from sklearn.datasets import make_moons, make_circles, make_blobs
import warnings ; warnings.filterwarnings('ignore')

# Dataset BankChurners (encodé - même pipeline que TD 2 à 5)
raw = pd.read_csv('BankChurners.csv')
data = raw[raw.columns[:-2]].copy()
data['churn'] = (data['Attrition_Flag']=='Attrited Customer').astype(int)
data['Gender'] = data['Gender'].map({'M':1, 'F':0})
data['Marital_Status'] =
data['Marital_Status'].map({'Single':0, 'Married':1, 'Divorced':2, 'Unknown':-1})
edu_map={'Unknown':0, 'Uneducated':1, 'High School':2, 'College':3, 'Graduate':4, 'Post-
Graduate':5, 'Doctorate':6}
inc_map={'Unknown':0, 'Less than $40K':1, '$40K - $60K':2, '$60K - $80K':3, '$80K -
$120K':4, '$120K +':5}
data['Education_Level'] = data['Education_Level'].map(edu_map)
data['Income_Category'] = data['Income_Category'].map(inc_map)
data = pd.get_dummies(data, columns=['Card_Category'], prefix='card',
drop_first=True, dtype=int)
feat_cols = [c for c in data.columns if c not in
['Attrition_Flag', 'churn', 'CLIENTNUM']]
X_b = data[feat_cols] ; y_b = data['churn']
X_tr_b, X_te_b, y_tr_b, y_te_b = train_test_split(X_b, y_b, test_size=0.2,
random_state=42, stratify=y_b)
print(f'BankChurners prêt : {X_tr_b.shape}')
```



Partie 1 — SVM Linéaire : Visualisation de la Marge

Durée estimée : 40 minutes | Voir et comprendre géométriquement la marge, l'hyperplan et les vecteurs support.

Q 1.1 Visualisation de la marge maximale — données 2D linéairement séparables

Consigne

Générez un dataset 2D linéairement séparable et visualisez la frontière de décision SVM avec sa marge et ses vecteurs support.

```
# Données synthétiques linéairement séparables
np.random.seed(42)
X_lin, y_lin = make_blobs(n_samples=80, centers=2,
                          cluster_std=0.8, random_state=42)
y_lin = (y_lin * 2 - 1) # Convertir en {-1, +1}

# SVM linéaire - hard margin (C très grand ≈ hard)
svm_lin = SVC(kernel='linear', C=1000.0)
svm_lin.fit(X_lin, y_lin)

# — Fonction de visualisation SVM —
def plot_svm_margin(svm, X, y, ax=None, title='SVM'):
    """Visualise l'hyperplan, la marge et les vecteurs support."""
    if ax is None:
        _, ax = plt.subplots(figsize=(9, 7))

    # Grille de prédiction
    x_min, x_max = X[:,0].min()-0.5, X[:,0].max()+0.5
    y_min, y_max = X[:,1].min()-0.5, X[:,1].max()+0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                          np.linspace(y_min, y_max, 300))
    Z = svm.decision_function(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Frontière et marges
    ax.contourf(xx, yy, Z, levels=20, cmap='RdYlGn', alpha=.4)
    ax.contour(xx, yy, Z, levels=[-1, 0, 1],
               colors=['#9C3D38', '#1A2550', '#1A7A6E'],
               linewidths=[1.5, 2.5, 1.5],
               linestyles=['--', '-', '--'])

    # Points de données
```



```
colors = np.where(y == 1, '#1A7A6E', '#9C3D38')
ax.scatter(X[:,0], X[:,1], c=colors, s=50,
           edgecolors='black', linewidths=0.8, zorder=5)

# Vecteurs support — entourés en or
sv = svm.support_vectors_
ax.scatter(sv[:,0], sv[:,1], s=200, facecolors='none',
           edgecolors='#B07D3A', linewidths=2.5, zorder=6,
           label=f'Vecteurs support ({len(sv)})')

# Calcul de la marge
w = svm.coef_[0]
margin = 2 / np.linalg.norm(w)
ax.set_title(f'{title}\nMarge = {margin:.3f} | C={svm.C} | {len(sv)} vecteurs support',
            fontweight='bold')
ax.legend(loc='upper right')
ax.set_xlabel('Feature 1') ; ax.set_ylabel('Feature 2')
return ax

fig, ax = plt.subplots(figsize=(10, 7))
plot_svm_margin(svm_lin, X_lin, y_lin, ax=ax, title='SVM Linéaire — Hard Margin')
plt.tight_layout() ; plt.show()

print(f'Vecteurs support : {svm_lin.n_support_}')
print(f'Coeffs w : {svm_lin.coef_[0].round(4)}')
print(f'Biais b : {svm_lin.intercept_[0]:.4f}')
print(f'Marge = 2/||w|| : {2/np.linalg.norm(svm_lin.coef_[0]):.4f}')
```

📖 Lire la visualisation SVM

- **Ligne centrale** (bleu foncé) : hyperplan de décision — $w \cdot x + b = 0$
- **Lignes pointillées** (vert et rouge) : hyperplans marginaux — $w \cdot x + b = \pm 1$
- **Zone entre les pointillés** : la marge — doit être la plus large possible
- **Cercles dorés** : vecteurs support — les seuls exemples qui définissent la frontière
- **Tous les autres exemples** pourraient être supprimés sans changer l'hyperplan

Questions d'analyse

1. Combien de vecteurs support y a-t-il ? Sont-ils proches ou loin de la frontière ?
2. Si vous supprimiez tous les exemples qui NE sont PAS vecteurs support, l'hyperplan changerait-il ? Testez en ré-entraînant sur les seuls vecteurs support.
3. Calculez manuellement la marge à partir des coefficients w : $\text{marge} = 2 / ||w||$. Correspond-elle à la valeur affichée ?



Q 1.2

Effet du paramètre C — Soft Margin

Consigne

Générez des données avec du chevauchement (non parfaitement séparables) et comparez visuellement les frontières pour $C = 0.01$, $C = 1$, $C = 100$.

```
# Données avec chevauchement (non parfaitement séparables)
X_soft, y_soft = make_blobs(n_samples=120, centers=2,
                             cluster_std=1.8, random_state=42)
y_soft = (y_soft * 2 - 1)

C_values = [0.01, 1, 100]
fig, axes = plt.subplots(1, 3, figsize=(19, 6))

for ax, C in zip(axes, C_values):
    svm_c = SVC(kernel='linear', C=C)
    svm_c.fit(X_soft, y_soft)
    plot_svm_margin(svm_c, X_soft, y_soft, ax=ax,
                    title=f'SVM Soft Margin - C={C}')

plt.suptitle('Impact du paramètre C sur la marge SVM',
             fontsize=14, fontweight='bold')
plt.tight_layout() ; plt.show()

# Tableau comparatif
print(f'{C:>8s}  {'Marge':>8s}  {'NB_SV':>8s}  {'Violations'}')
print('-' * 50)
for C in [0.01, 0.1, 1, 10, 100, 1000]:
    svm = SVC(kernel='linear', C=C).fit(X_soft, y_soft)
    w = svm.coef_[0]
    margin = 2 / np.linalg.norm(w)
    # Violations = exemples dans la marge ou mal classés
    dec = svm.decision_function(X_soft)
    violations = np.sum(np.abs(dec) < 1)
    print(f'{C:>8.2f}  {margin:>8.4f}  {svm.n_support_.sum():>8d}  {violations}')
```

📌 Rappel de cours — Soft Margin — Interprétation des Variables d'Écart (Slack variables ξ_i)

- Le SVM soft margin minimise : $\frac{1}{2} \|w\|^2 + C \cdot \sum \xi_i$ sous $y_i(w \cdot x_i + b) \geq 1 - \xi_i$
- $\xi_i = 0$: exemple correctement classé, hors de la marge
- $0 < \xi_i \leq 1$: exemple dans la marge, mais du bon côté



- $\xi_i > 1$: exemple mal classé (erreur de classification)
- **C grand** : le terme $C \cdot \sum \xi_i$ pèse plus $\rightarrow$ moins de violations tolérées $\rightarrow$ marge étroite
- **C petit** : le terme $\|w\|^2/2$ pèse plus $\rightarrow$ marge large $\rightarrow$ plus de violations tolérées

Questions d'analyse

4. Avec $C=0.01$, la marge est-elle plus grande ou plus petite qu'avec $C=100$? Combien de vecteurs support y a-t-il dans chaque cas ?
5. Avec C très grand (hard margin approximatif), des exemples sont-ils encore mal classés ? Qu'est-ce que cela révèle sur la séparabilité de ces données ?
6. Sur un dataset avec du bruit, quel C préférez-vous ? Argumentez en termes de biais/variance et de robustesse.
7. **Connexion avec la régularisation** : le C du SVM est-il analogue au C du LogisticRegression (TD4) ? Comparez les deux paramètres.



Partie 2 — Le Kernel Trick : Séparation Non-Linéaire

Durée estimée : 40 minutes | Données en lune, cercles concentriques — impossible à séparer linéairement, trivial avec un noyau.

Noyau RBF (Radial Basis Function) — le plus courant

$$K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \cdot \|\mathbf{x} - \mathbf{y}\|^2) \quad \text{avec } \gamma > 0$$

Q 2.1

Comparaison linéaire vs RBF — données en lune

Consigne

Testez un SVM linéaire et un SVM RBF sur le dataset 'make_moons' (non-linéairement séparable). Visualisez et comparez les frontières.

```
# Dataset 'moons' — archétype des données non-linéaires
X_moon, y_moon = make_moons(n_samples=300, noise=0.2, random_state=42)
X_m_tr, X_m_te, y_m_tr, y_m_te = train_test_split(
    X_moon, y_moon, test_size=0.25, random_state=42)

# Normalisation — obligatoire pour SVM
scaler_m = StandardScaler()
X_m_tr_sc = scaler_m.fit_transform(X_m_tr)
X_m_te_sc = scaler_m.transform(X_m_te)

# Comparaison des noyaux
kernels = {
    'Linéaire': SVC(kernel='linear', C=1),
    'RBF (C=1)': SVC(kernel='rbf', C=1, gamma='scale'),
    'Polynomial 3': SVC(kernel='poly', C=1, degree=3, gamma='scale'),
}

fig, axes = plt.subplots(1, 3, figsize=(19, 6))

for ax, (name, svm) in zip(axes, kernels.items()):
    svm.fit(X_m_tr_sc, y_m_tr)
    score = svm.score(X_m_te_sc, y_m_te)

# Grille
x_min, x_max = X_m_tr_sc[:,0].min()-0.3, X_m_tr_sc[:,0].max()+0.3
y_min, y_max = X_m_tr_sc[:,1].min()-0.3, X_m_tr_sc[:,1].max()+0.3
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
```



```
np.linspace(y_min, y_max, 300))
Z = svm.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdYlGn')
ax.scatter(X_m_tr_sc[:, 0], X_m_tr_sc[:, 1],
           c=['#1A7A6E' if y==1 else '#9C3D38' for y in y_m_tr],
           s=25, edgecolors='black', linewidths=0.5, alpha=0.7)

# Vecteurs support (uniquement pour les noyaux non-linéaires)
if name != 'Linéaire':
    sv = svm.support_vectors_
    ax.scatter(sv[:, 0], sv[:, 1], s=100, facecolors='none',
              edgecolors='#B07D3A', linewidths=2, alpha=0.8)

ax.set_title(f'{name}\nAccuracy test = {score:.3f} ({svm.n_support_.sum()} SV)',
             fontweight='bold')
ax.set_xlabel('Feature 1 (normalisée)')

plt.suptitle('Comparaison des Noyaux SVM - Données Moons',
             fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```

Questions d'analyse

8. Le SVM linéaire arrive-t-il à séparer les données en lune ? Quelle accuracy obtient-il ? Ce résultat était-il attendu ?
9. Le SVM RBF produit-il une frontière courbe ? Quel est le gain d'accuracy par rapport au SVM linéaire ?
10. Le SVM polynomial (degré 3) est-il meilleur ou moins bon que le RBF sur ce dataset ? À quoi attribuez-vous la différence ?

Q 2.2

Données en cercles concentriques — SVM RBF vs Linéaire

Consigne

Testez le même protocole sur `make_circles` — un cas encore plus difficile pour un SVM linéaire.

```
# Cercles concentriques — séparation linéaire impossible
X_circ, y_circ = make_circles(n_samples=300, noise=0.15,
                             factor=0.4, random_state=42)
X_c_tr, X_c_te, y_c_tr, y_c_te = train_test_split(
    X_circ, y_circ, test_size=0.25, random_state=42)
```



```
sc_c = StandardScaler()
X_c_tr_sc = sc_c.fit_transform(X_c_tr)
X_c_te_sc = sc_c.transform(X_c_te)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for ax, (kernel, C, gamma, label) in zip(axes, [
    ('linear', 10, 'scale', 'Linéaire'),
    ('rbf', 1, 'scale', 'RBF'),
]):
    svm = SVC(kernel=kernel, C=C, gamma=gamma)
    svm.fit(X_c_tr_sc, y_c_tr)
    score = svm.score(X_c_te_sc, y_c_te)

    xx, yy = np.meshgrid(np.linspace(-2,2,300), np.linspace(-2,2,300))
    Z = svm.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdYlGn')
    ax.scatter(X_c_tr_sc[:,0], X_c_tr_sc[:,1],
               c=['#1A7A6E' if y==1 else '#9C3D38' for y in y_c_tr],
               s=30, edgecolors='black', linewidths=0.7)
    ax.set_title(f'SVM {label}\nAccuracy = {score:.3f}', fontweight='bold')

plt.suptitle('Cercles Concentriques - SVM Linéaire vs RBF',
             fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```

Comprendre le noyau RBF intuitivement

- Le noyau RBF mesure la **similarité gaussienne** entre deux points
- Dans l'espace de projection implicite, des exemples proches ($\|x-y\|$ petit) sont similaires ($K \approx 1$), des exemples éloignés ne le sont pas ($K \approx 0$)
- Pour les **cercles concentriques**, le noyau RBF projette les données dans un espace où elles deviennent linéairement séparables par une sphère
- C'est pour cela que RBF est souvent le noyau par défaut : il gère les patterns circulaires, en spirale, et de nombreuses géométries complexes

Questions d'analyse

11. Sur les cercles, l'accuracy du SVM linéaire est-elle proche du hasard (50%) ? Pourquoi est-ce théoriquement impossible de séparer des cercles avec une droite ?
12. **Réflexion mathématique** : pour les cercles dans $\mathbb{R}^2$, quelle transformation $\phi(x_1, x_2) = (x_1^2, \sqrt{2} \cdot x_1 x_2, x_2^2)$ ferait que les cercles deviennent séparables dans $\mathbb{R}^3$? Est-ce lié au noyau polynomial de degré 2 ?



Q 2.3

Effet de gamma sur le noyau RBF — biais/variance

Consigne

Visualisez l'impact de gamma (0.01, 1, 10, 100) sur la frontière de décision RBF. Identifiez sous-apprentissage et sur-apprentissage.

```
gammas = [0.01, 1, 10, 100]
fig, axes = plt.subplots(2, 2, figsize=(14, 11))
axes = axes.flatten()

for ax, gamma in zip(axes, gammas):
    svm_g = SVC(kernel='rbf', C=1.0, gamma=gamma)
    svm_g.fit(X_m_tr_sc, y_m_tr)
    score_tr = svm_g.score(X_m_tr_sc, y_m_tr)
    score_te = svm_g.score(X_m_te_sc, y_m_te)

    xx, yy = np.meshgrid(np.linspace(-3,3,300), np.linspace(-3,3,300))
    Z = svm_g.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.4, cmap='RdYlGn')
    ax.scatter(X_m_tr_sc[:,0], X_m_tr_sc[:,1],
               c=['#1A7A6E' if y==1 else '#9C3D38' for y in y_m_tr],
               s=20, edgecolors='black', linewidths=0.5, alpha=0.7)

    diag = 'SOUS-appr.' if score_tr < 0.85 else ('SUR-appr.' if score_tr - score_te >
0.1 else 'OPTIMAL')
    ax.set_title(f'γ={gamma} Train={score_tr:.3f} Test={score_te:.3f} [{diag}]',
                fontweight='bold')

plt.suptitle('Impact de gamma sur le Noyau RBF (C=1, Moons dataset)',
             fontsize=13, fontweight='bold')
plt.tight_layout() ; plt.show()
```

Questions d'analyse

13. Avec $\gamma=0.01$, la frontière est-elle très lisse ou très complexe ? Quel est l'effet sur les scores train/test ?
14. Avec $\gamma=100$, la frontière 'entoure' chaque exemple individuellement. Quel est le score train ? Et le score test ? Quel problème cela illustre-t-il ?
15. Quel γ offre le meilleur équilibre train/test sur le dataset Moons ? Comparez avec la valeur 'scale' de sklearn ($\gamma = 1/(n_{\text{features}} \times X.\text{var}())$).



Partie 3 — Optimisation de C et Gamma par GridSearchCV

Durée estimée : 30 minutes | Les deux hyperparamètres C et gamma interagissent — explorer leur espace conjoint.

Q 3.1

GridSearchCV 2D — Heatmap C × gamma

Consigne

Effectuez un GridSearch sur C et gamma sur le dataset Moons. Visualisez les résultats sous forme de heatmap pour comprendre l'interaction entre les deux paramètres.

```
from sklearn.model_selection import GridSearchCV

# Grille de recherche
param_grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1, 10],
}

grid_search = GridSearchCV(
    SVC(kernel='rbf'),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1
)
grid_search.fit(X_m_tr_sc, y_m_tr)

print(f'Meilleurs paramètres : {grid_search.best_params_}')
print(f'Meilleur score CV : {grid_search.best_score_:.4f}')

# Heatmap des scores CV
scores = grid_search.cv_results_['mean_test_score'].reshape(
    len(param_grid['C']), len(param_grid['gamma']))

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Heatmap
sns.heatmap(scores, annot=True, fmt='.3f', cmap='YlOrRd',
            xticklabels=param_grid['gamma'],
            yticklabels=param_grid['C'],
            ax=axes[0])
axes[0].set_xlabel('gamma') ; axes[0].set_ylabel('C')
```



```
axes[0].set_title('F1 macro (CV=5) - GridSearch SVM RBF', fontweight='bold')

# Frontière avec les meilleurs paramètres
best_svm = grid_search.best_estimator_
xx, yy = np.meshgrid(np.linspace(-3,3,300), np.linspace(-3,3,300))
Z = best_svm.predict(np.c_[xx.ravel(),yy.ravel()]).reshape(xx.shape)
axes[1].contourf(xx, yy, Z, alpha=0.4, cmap='RdYlGn')
axes[1].scatter(X_m_te_sc[:,0], X_m_te_sc[:,1],
                c=['#1A7A6E' if y==1 else '#9C3D38' for y in y_m_tel],
                s=30, edgecolors='black', linewidths=0.7)
axes[1].set_title(f'Meilleur SVM : C={grid_search.best_params_["C"]},
                  γ={grid_search.best_params_["gamma"]}',
                  fontweight='bold')
plt.tight_layout() ; plt.show()
```

❗ Piège fréquent — GridSearchCV et data leakage

Le **StandardScaler doit être dans un Pipeline avec le SVC** lorsqu'on utilise GridSearchCV. Sans Pipeline, le scaler est fitté sur l'ensemble de X_{train} (y compris les folds de validation) → data leakage subtil.

Toujours écrire :

```
# CORRECT — Pipeline dans GridSearchCV
pipe_svm = Pipeline([('sc', StandardScaler()), ('svc', SVC(kernel='rbf'))])
param_grid_pipe = {'svc__C': [0.1, 1, 10], 'svc__gamma': [0.01, 0.1, 1]}
grid = GridSearchCV(pipe_svm, param_grid_pipe, cv=5, scoring='f1')
grid.fit(X_tr, y_tr)
```

Questions d'analyse

16. Sur la heatmap, y a-t-il une zone (C, gamma) clairement supérieure aux autres ? Décrivez la forme de cette zone.
17. Que se passe-t-il dans le coin supérieur droit de la heatmap (C grand et gamma grand) ? Et dans le coin inférieur gauche ?
18. Pourquoi est-il important d'utiliser un Pipeline dans GridSearchCV plutôt que de scaler avant ? Quel risque cela représente-t-il sur l'estimation des performances ?

Q 3.2

Courbe de validation — C avec gamma fixé

```
# Fixer gamma au meilleur et faire varier C
best_gamma = grid_search.best_params_['gamma']

C_range = np.logspace(-3, 3, 15)
```



```
pipe_vc = Pipeline([('sc', StandardScaler()),
                    ('svc', SVC(kernel='rbf', gamma=best_gamma))])

train_sc, val_sc = validation_curve(
    pipe_vc, X_m_tr_sc, y_m_tr,
    param_name='svc__C',
    param_range=C_range,
    cv=5, scoring='f1_macro', n_jobs=-1
)

plt.figure(figsize=(11, 5))
plt.semilogx(C_range, train_sc.mean(axis=1), 'o-', color='#1A7A6E', label='F1 train')
plt.semilogx(C_range, val_sc.mean(axis=1), 's-', color='#9C3D38', label='F1 val
CV')
plt.fill_between(C_range,
                 val_sc.mean(axis=1) - val_sc.std(axis=1),
                 val_sc.mean(axis=1) + val_sc.std(axis=1),
                 alpha=.2, color='#9C3D38')
plt.axvline(grid_search.best_params_['C'], color='#B07D3A', ls='--', lw=2,
            label=f'C optimal={grid_search.best_params_["C"]}')
plt.xlabel('C (échelle log)') ; plt.ylabel('F1 macro')
plt.title(f'Courbe de Validation SVM RBF ( $\gamma$ = {best_gamma})',
          fontweight='bold')
plt.legend() ; plt.tight_layout() ; plt.show()
```

Questions d'analyse

19. La courbe de validation a-t-elle la forme typique en U (ou cloche inversée) ? Identifiez les zones under/overfitting.
20. L'intervalle de confiance (zone ombrée) est-il large ou étroit ? Qu'est-ce que cela indique sur la stabilité du modèle ?



Partie 4 — SVM Appliqué sur BankChurners

Durée estimée : 30 minutes | Passer d'un dataset 2D jouet à un vrai problème bancaire — réalité et compromis.

Q 4.1

Premier SVM sur BankChurners — pipeline complet

Consigne

Entraînez un SVM RBF sur BankChurners avec la Pipeline correcte (scaler + SVC). Évaluez avec les métriques adaptées au déséquilibre de classes.

```
# Pipeline : StandardScaler + SVC
pipe_svm_b = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', C=1.0, gamma='scale',
                probability=True, # nécessaire pour ROC/proba
                random_state=42))
])

pipe_svm_b.fit(X_tr_b, y_tr_b)
y_pred_svm = pipe_svm_b.predict(X_te_b)
y_prob_svm = pipe_svm_b.predict_proba(X_te_b)[:, 1]

print('=== SVM RBF - BankChurners (C=1, gamma=scale) ===')
print(classification_report(y_te_b, y_pred_svm,
                            target_names=['Existing (0)', 'Attrited (1)']))
print(f'AUC-ROC : {roc_auc_score(y_te_b, y_prob_svm):.4f}')

# Temps d'entraînement
import time
t0 = time.time()
pipe_svm_b.fit(X_tr_b, y_tr_b)
t_svm = time.time() - t0
print(f'Temps entraînement SVM : {t_svm:.2f}s')
```

📌 SVM sur grands datasets — mise en garde

- La complexité d'entraînement du SVM est entre $O(n^2d)$ et $O(n^3d)$ — très lent pour $n > 10\,000$
- BankChurners avec 10 127 exemples est à la limite — le SVM peut prendre plusieurs minutes
- Pour de très grands datasets, utiliser **LinearSVC** (optimisation liblinear, $O(n)$) ou **SGDClassifier(loss='hinge')** en lieu et place de SVC



- Le paramètre **probability=True** est requis pour `predict_proba` mais ralentit encore le SVM (calibration Platt)

Questions d'analyse

21. Comparez l'AUC-ROC du SVM avec celui de la Régression Logistique (TD4) et de l'Arbre de Décision (TD5). Le SVM est-il meilleur ?
22. Le temps d'entraînement du SVM est-il comparable à celui de la Régression Logistique ? Qu'est-ce que cela implique pour son utilisation en production ?
23. Pour ce dataset avec ~10 000 exemples, quelle alternative au SVC plein recommanderiez-vous pour un déploiement en production ?

Q 4.2

GridSearch C et gamma sur BankChurners

Consigne

Optimisez les hyperparamètres C et gamma du SVM RBF sur BankChurners. Utilisez un `scoring='f1'` adapté au déséquilibre de classes.

```
# GridSearch sur un sous-ensemble pour la rapidité
# (ou utiliser RandomizedSearchCV pour un espace plus large)
param_grid_b = {
    'svc__C': [0.1, 1, 10],
    'svc__gamma': ['scale', 0.01, 0.1],
}

pipe_gs = Pipeline([
    ('scaler', StandardScaler()),
    ('svc', SVC(kernel='rbf', probability=True, random_state=42))
])

grid_b = GridSearchCV(
    pipe_gs, param_grid_b,
    cv=3, # 3-fold pour réduire le temps
    scoring='f1',
    n_jobs=-1,
    verbose=1
)
grid_b.fit(X_tr_b, y_tr_b)

print(f'Meilleurs paramètres : {grid_b.best_params_}')
print(f'Meilleur F1 CV : {grid_b.best_score_:.4f}')
```



```
# Évaluation finale
y_best_b = grid_b.best_estimator_.predict(X_te_b)
y_prob_b = grid_b.best_estimator_.predict_proba(X_te_b)[: ,1]
print('\n=== SVM OPTIMAL – Évaluation finale ===')
print(classification_report(y_te_b, y_best_b,
    target_names=['Existing (0)', 'Attrited (1)']))
print(f'AUC-ROC : {roc_auc_score(y_te_b, y_prob_b):.4f}')
```

Q 4.3

Benchmark final — 4 modèles comparés

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

# Tous les modèles avec pipeline scaler
all_models = {
    'Régression Logistique': Pipeline([
        ('sc', StandardScaler()),
        ('m', LogisticRegression(C=1, max_iter=1000, random_state=42))
    ]),
    'Arbre de Décision': Pipeline([
        ('m', DecisionTreeClassifier(max_depth=8, random_state=42))
    ]),
    'KNN (K=7)': Pipeline([
        ('sc', StandardScaler()),
        ('m', KNeighborsClassifier(n_neighbors=7))
    ]),
    'SVM RBF (optimal)': grid_b.best_estimator_,
}

print('===== BENCHMARK FINAL – TD 1-6 =====')
print(f" {'Modèle':25s} {'F1 CV (5)':>12s} {'±':>6s} {'F1 Test':>10s} {'AUC':>8s}")
print(' ' + '-'*65)

for name, model in all_models.items():
    cv_scores = cross_val_score(model, X_tr_b, y_tr_b,
                                cv=5, scoring='f1', n_jobs=-1)

    model.fit(X_tr_b, y_tr_b)
    y_p = model.predict(X_te_b)
    f1_t = f1_score(y_te_b, y_p)
    try:
        proba = model.predict_proba(X_te_b)[: ,1]
        auc = roc_auc_score(y_te_b, proba)
    except:
        auc = float('nan')
```



Questions d'analyse

24. Après 6 TDs, quel modèle donne les **meilleurs résultats** sur BankChurners ? Le résultat vous surprend-il ?
25. Le SVM est-il compétitif avec les autres modèles sur ce dataset ? Justifiez par les métriques ET par les contraintes pratiques (temps, déploiement, explicabilité).
26. La Régression Logistique (le modèle le plus simple) est-elle très distancée par les modèles complexes ? Que dit le **principe d'Occam** sur ce résultat ?



Section Bonus — Pour aller plus loin

Ces exercices approfondissent la SVR (régression avec SVM) et la classification multi-classe.

Bonus B1 — SVR : Régression par SVM

Le SVM peut aussi faire de la régression ! Le SVR (Support Vector Regressor) trouve un tube d'épaisseur ϵ autour de la fonction cible.

```
from sklearn.svm import SVR
from sklearn.datasets import fetch_california_housing

# Dataset California Housing pour la régression
housing = fetch_california_housing(as_frame=True)
X_h = housing.frame[['MedInc', 'HouseAge', 'AveRooms', 'AveOccup']]
y_h = housing.target

X_h_tr, X_h_te, y_h_tr, y_h_te = train_test_split(
    X_h, y_h, test_size=0.2, random_state=42)

pipe_svr = Pipeline([
    ('sc', StandardScaler()),
    ('svr', SVR(kernel='rbf', C=10, gamma='scale', epsilon=0.1))
])
pipe_svr.fit(X_h_tr, y_h_tr)

from sklearn.metrics import mean_squared_error, r2_score
y_svr_pred = pipe_svr.predict(X_h_te)
print(f'SVR R2 : {r2_score(y_h_te, y_svr_pred):.4f}')
print(f'SVR RMSE : {np.sqrt(mean_squared_error(y_h_te, y_svr_pred)):.4f}')

# Comparer avec LinearRegression
from sklearn.linear_model import LinearRegression
lr_h = Pipeline([('sc', StandardScaler()), ('lr', LinearRegression())])
lr_h.fit(X_h_tr, y_h_tr)
y_lr_pred = lr_h.predict(X_h_te)
print(f'LR R2 : {r2_score(y_h_te, y_lr_pred):.4f}')
print(f'LR RMSE : {np.sqrt(mean_squared_error(y_h_te, y_lr_pred)):.4f}')
```

27. Le SVR obtient-il un R^2 supérieur à la Régression Linéaire ? Cette différence justifie-t-elle le temps d'entraînement plus long ?
28. Le paramètre **epsilon** (tube d'insensibilité) est un hyperparamètre supplémentaire de SVR. Testez $\epsilon \in \{0.01, 0.1, 0.5\}$: quel impact sur le R^2 ?



Bonus B2 — Classification Multi-classe : OvO vs OvR

sklearn utilise par défaut OvO (One vs One) pour SVC. Comparons avec OvR (One vs Rest) sur le dataset Iris à 3 classes.

```
from sklearn.datasets import load_iris
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier

iris = load_iris()
X_iris, y_iris = iris.data, iris.target
X_i_tr, X_i_te, y_i_tr, y_i_te = train_test_split(
    X_iris, y_iris, test_size=0.25, random_state=42, stratify=y_iris)

strategies = {
    'SVC (OvO auto)': SVC(kernel='rbf', C=10, gamma='scale'),
    'OvO explicite': OneVsOneClassifier(SVC(kernel='rbf', C=10)),
    'OvR explicite': OneVsRestClassifier(SVC(kernel='rbf', C=10)),
}

print('=== MULTI-CLASSE SVM - IRIS (3 classes) ===')
print(f"{'Stratégie':25s} {'Accuracy':>10s} {'N classifieurs':>16s}")
for name, model in strategies.items():
    sc = StandardScaler()
    X_sc = sc.fit_transform(X_i_tr)
    X_te_sc = sc.transform(X_i_te)
    model.fit(X_sc, y_i_tr)
    acc = model.score(X_te_sc, y_i_te)
    n_clf = getattr(model, 'estimators_', None)
    n = len(n_clf) if n_clf else 'N/A'
    print(f'{name:25s} {acc:>10.4f} {str(n):>16s}')
```



Livrable attendu

✓ Ce que vous devez remettre

- Notebook **TD6_NomPrenom.ipynb** — toutes les cellules exécutées avec outputs visibles
- **Visualisation de la marge** avec vecteurs support sur données 2D, valeur de marge calculée
- **Tableau comparatif** C petit / C=1 / C grand avec marges et nombre de vecteurs support
- **Comparaison des noyaux** (linéaire, RBF, polynomial) sur Moons et/ou Circles
- **Heatmap GridSearch** C × gamma avec identification du couple optimal
- **Benchmark final** des 4 modèles (RL + Arbre + KNN + SVM) sur BankChurners
- Une cellule Markdown de **conclusion** (~200 mots) : quand utiliser SVM, forces et limites observées
- Délai : **avant le TD 7**, dépôt sur la plateforme pédagogique

Critères d'évaluation

Critère	Coefficient	Description
Visualisation marge + vecteurs support	20%	Clarté, valeur calculée correcte
Analyse de l'effet du paramètre C	20%	Tableau, interprétation biais/variance
Comparaison des noyaux + explications	25%	Graphiques, interprétation géométrique
GridSearch + heatmap C×gamma	20%	Pipeline correct, analyse de la heatmap
Section Bonus (optionnel)	+ 15%	SVR ou multi-classe OvO/OvR

Ressources et Documentation

- **sklearn** — **SVC / SVM** : <https://scikit-learn.org/stable/modules/svm.html>
- **sklearn** — **GridSearchCV** : https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- **SVMLight** (Joachims, original SVM) : <https://svmlight.joachims.org/>
- **A Practical Guide to SVM (Hsu et al.)** : <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf>
- **sklearn** — **SVR** : <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html>



Paramètres clés de SVC à retenir

Paramètre	Valeur défaut	Rôle et impact
kernel	'rbf'	Noyau utilisé : 'linear', 'poly', 'rbf', 'sigmoid'
C	1.0	Inverse de la régularisation. Grand C → moins de marge → moins d'erreurs train
gamma	'scale'	Influence RBF/poly. 'scale'=1/(n_feat×var). Grand γ → sur-apprentissage
degree	3	Degré du noyau polynomial
probability	False	Activer pour predict_proba (plus lent — calibration Platt)
class_weight	None	'balanced' pour données déséquilibrées
decision_function_shape	'ovr'	Multi-classe : 'ovr' (défaut sklearn 0.19+) ou 'ovo'